

NYC Taxi Data Pipeline

A **production-grade ETL (Extract-Transform-Load) pipeline** that processes 500K+ NYC Yellow Taxi trip records from January 2023, transforms raw data, loads it into a PostgreSQL database, and derives actionable insights through SQL analytics.

Portfolio Project | Data Engineering | Python + SQL + PostgreSQL

Project Overview

This pipeline demonstrates **end-to-end data engineering skills** by ingesting real-world NYC taxi data, applying data quality transformations, persisting to a cloud database, and executing analytical queries to uncover business insights.

Key Metrics

- **Dataset:** 2.8M rows (500K processed) | 17 columns
- **Data Format:** Parquet (columnar, compressed)
- **Processing:** Python (pandas, SQLAlchemy) + SQL
- **Database:** Neon PostgreSQL (cloud)
- **Insights Derived:** 5 analytical queries with actionable findings

Key Findings

Query	Insight
Peak Hours	5 PM is NYC's busiest commute hour (17:00)
Revenue Surge	Jan 1, 2023 generated \$4.47M (New Year's effect)
Busiest Zones	Midtown Manhattan dominates pickup locations
Late-Night Trips	Longer average distances during 11 PM–5 AM
Trip Duration	Consistent ~13-15 min average across days

Tech Stack

Layer	Technology
Data Source	NYC Yellow Taxi (Parquet format)
Processing	Python 3.9+ (pandas, pyarrow, SQLAlchemy)
Database	PostgreSQL (Neon cloud)
Analytics	SQL (window functions, aggregations)
Logging	Python logging module
Environment	Kaggle notebooks + local reproducibility

Project Structure

nyc-taxi-pipeline/

```
|— src/                # Production Python code
| |— config.py         # Configuration & secrets
| |— logger.py         # Logging setup
| |— extract.py        # Data extraction
| |— transform.py      # Data transformations
| |— load.py           # Database loading
| |— pipeline.py       # Orchestration
|
|— sql/                # Database & analytics
| |— schema.sql        # Table schema
```

```
| └─ analysis/          # SQL queries
|
|─ notebooks/          # Jupyter (demo + exploration)
|   └─ 00_project_overview.ipynb
|   └─ 01_extract_data.ipynb
|   └─ 02_transform_data.ipynb
|   └─ 03_load_database.ipynb
|   └─ 04_analysis_queries.ipynb
|
|─ outputs/            # Results, visualizations, CSV exports
|─ docs/               # Detailed documentation
|─ tests/              # Unit tests
|─ requirements.txt     # Dependencies
|─ .env.example         # Environment template
└─ README.md           # This file
```

Quick Start

Prerequisites

- Python 3.9+
- PostgreSQL account (or Neon free tier)
- Git

1 Clone & Install

git clone <https://github.com/yourusername/nyc-taxi-pipeline.git>

```
cd nyc-taxi-pipeline
```

```
pip install -r requirements.txt
```

2 Configure Credentials

Create `.env` file from template:

```
cp .env.example .env
```

Edit `.env` with your PostgreSQL credentials:

```
DATABASE_URL=postgresql://user:password@host:5432/dbname?sslmode=require
```

```
PARQUET_PATH=/path/to/yellow_tripdata_2023-01.parquet
```

```
TABLE_NAME=yellow_taxi_2023
```

```
SAMPLE_SIZE=500000
```

3 Run the Pipeline

```
python -m src.pipeline
```

Output:

```
2024-01-15 10:23:45 - INFO - ✓ Extracted 500,000 rows
```

```
2024-01-15 10:23:47 - INFO - ✓ Transformed 500,000 rows
```

```
2024-01-15 10:23:52 - INFO - ✓ Loaded 500,000 rows to PostgreSQL
```

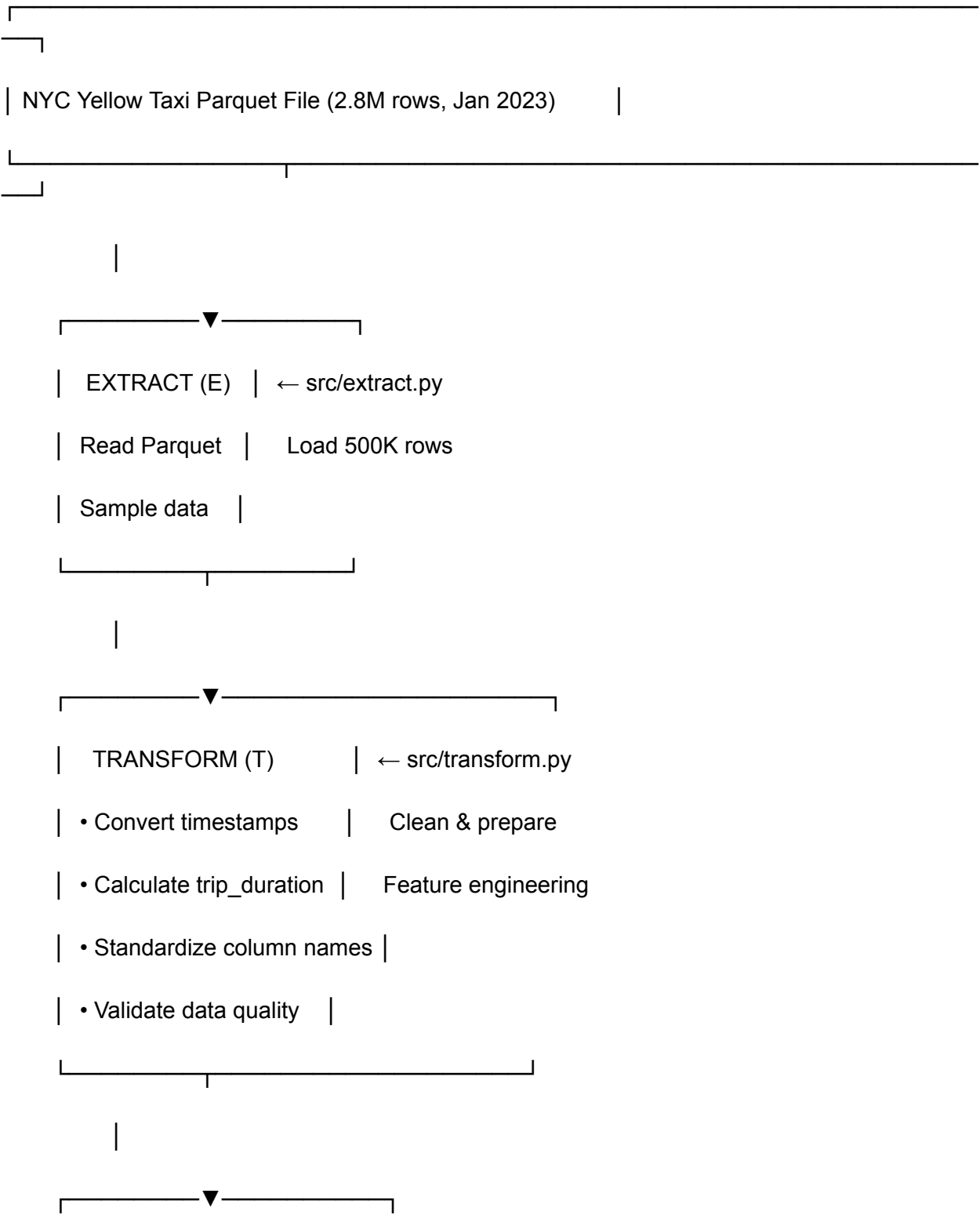
```
2024-01-15 10:23:53 - INFO - Pipeline completed successfully!
```

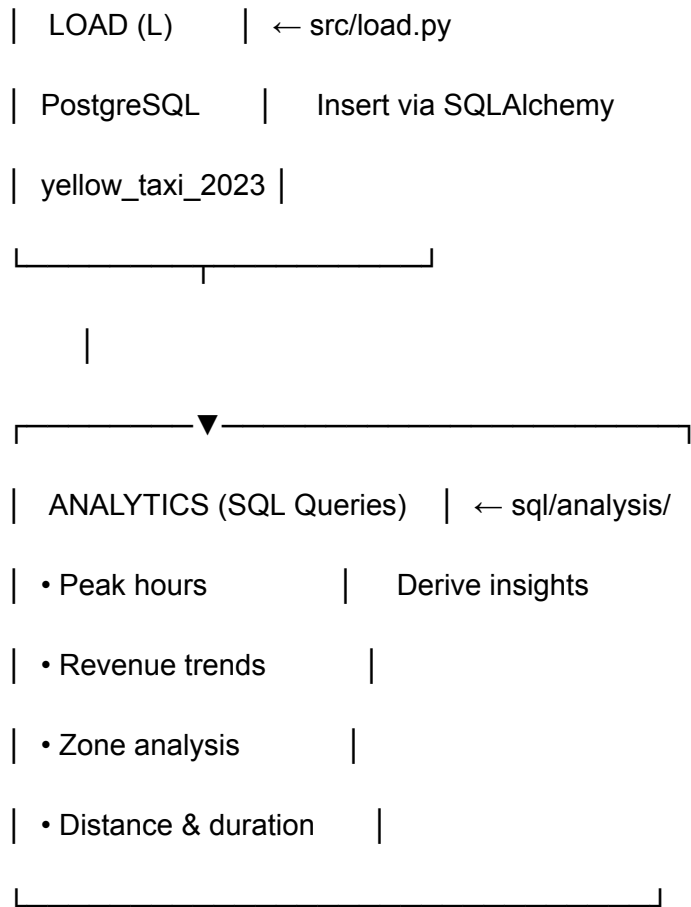
4 Run Analytics

```
# Open Jupyter notebook with pre-built queries
```

```
jupyter notebook notebooks/04_analysis_queries.ipynb
```

Data Flow





ETL Details

Extract (E)

What: Read NYC Yellow Taxi Parquet file

```
def extract_data(path: str, sample_size: int) -> pd.DataFrame:
```

```
    """Load Parquet file with optional sampling."""
```

```
    df = pd.read_parquet(path)
```

```
    return df.head(sample_size)
```

- **Input:** `yellow_tripdata_2023-01.parquet` (2.8M rows)
- **Output:** DataFrame (500K rows, 17 columns)

- **Why sample?** Neon PostgreSQL free tier has 512 MB storage limit

Transform (T)

What: Clean, enrich, and standardize data

```
def transform_data(df: pd.DataFrame) -> pd.DataFrame:
```

```
    """Apply data quality transformations."""
```

```
    # Convert timestamps
```

```
    df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'])
```

```
    df['tpep_dropoff_datetime'] = pd.to_datetime(df['tpep_dropoff_datetime'])
```

```
    # Create trip duration
```

```
    df['trip_duration'] = (
        df['tpep_dropoff_datetime'] - df['tpep_pickup_datetime']
    ).dt.total_seconds() / 60
```

```
    # Standardize column names
```

```
    df = df.rename(columns={
        "VendorID": "vendor_id",
        "RatecodeID": "rate_code_id",
        "PULocationID": "pu_location_id",
        "DOLocationID": "do_location_id"
    })
```

return df

- **Transformations:**

- Convert timestamps to datetime
- Calculate trip_duration (seconds → minutes)
- Rename columns (PascalCase → snake_case)
- Handle missing values
- Validate data types

Load (L)

What: Persist transformed data to PostgreSQL

def load_to_database(df: pd.DataFrame, table_name: str) -> None:

```
    """Insert DataFrame into PostgreSQL table."""
```

```
    engine = create_engine(DATABASE_URL)
```

```
    df.to_sql(table_name, engine, if_exists="append", index=False)
```

- **Database:** Neon PostgreSQL
- **Table:** `yellow_taxi_2023`
- **Rows inserted:** 500,000
- **Schema:** Auto-generated by SQLAlchemy

SQL Analytics

1. Peak Hours

```
SELECT EXTRACT(HOUR FROM tpep_pickup_datetime) AS hour,
```

```
       COUNT(*) AS trips
```

```
FROM yellow_taxi_2023
```

```
GROUP BY hour
```

```
ORDER BY trips DESC;
```

Insight: 5 PM (17:00) is peak—NYC commute hour with highest demand.

2. Revenue Per Day

```
SELECT DATE(tpcp_pickup_datetime) AS day,
```

```
        SUM(total_amount) AS revenue
```

```
FROM yellow_taxi_2023
```

```
GROUP BY day
```

```
ORDER BY day;
```

Insight: Jan 1, 2023 = \$4.47M (New Year's surge in demand).

3. Busiest Pickup Zones

```
SELECT pu_location_id,
```

```
        COUNT(*) AS trips
```

```
FROM yellow_taxi_2023
```

```
GROUP BY pu_location_id
```

```
ORDER BY trips DESC
```

```
LIMIT 10;
```

Insight: Midtown Manhattan dominates (locations 237, 236, 142).

4. Average Trip Distance Per Hour

```
SELECT EXTRACT(HOUR FROM tpep_pickup_datetime) AS hour,
```

```
        AVG(trip_distance) AS avg_distance
```

```
FROM yellow_taxi_2023
```

```
GROUP BY hour
```

```
ORDER BY hour;
```

Insight: Late-night trips (11 PM–5 AM) are ~30% longer on average.

5. Average Trip Duration Per Day

```
SELECT DATE(tpep_pickup_datetime) AS day,
```

```
       AVG(trip_duration) AS avg_duration
```

```
FROM yellow_taxi_2023
```

```
GROUP BY day
```

```
ORDER BY day;
```

Insight: Consistent 13–15 minute average across all days.

Testing

Run unit tests to verify data quality:

pytest tests/

Tests cover:

- ✓ Extract: File reading, sampling logic
 - ✓ Transform: Timestamp conversion, column renaming, data validation
 - ✓ Load: Database connection, insertion success
-

Documentation

- [ARCHITECTURE.md](#) — System design & flow diagrams
 - [SETUP.md](#) — Detailed setup for different environments
 - [DATA_DICTIONARY.md](#) — Column definitions & transformations
 - [QUERIES.md](#) — Explanation of all SQL queries
 - [TROUBLESHOOTING.md](#) — Common issues & fixes
-

Skills Demonstrated

Data Engineering

- ✓ **ETL Pipelines** — End-to-end Extract-Transform-Load
- ✓ **Data Quality** — Validation, cleaning, null handling
- ✓ **Cloud Databases** — PostgreSQL (Neon free tier)
- ✓ **Scalable Code** — Modular, reusable functions
- ✓ **Logging & Error Handling** — Production-ready patterns

Data Analysis & SQL

- ✓ **Analytical Queries** — Aggregations, window functions, date functions
- ✓ **Data Insights** — Peak hours, revenue trends, zone analysis
- ✓ **SQL Optimization** — Efficient GROUP BY, indexing awareness

Python & Tools

- ✓ **pandas** — Data manipulation (500K rows)
- ✓ **SQLAlchemy** — ORM & database abstraction
- ✓ **Configuration Management** — Environment-based config
- ✓ **Logging** — Structured logging for debugging
- ✓ **Jupyter Notebooks** — Exploratory analysis & documentation

DevOps & Best Practices

- ✓ **Git & Version Control** — Professional repository structure
- ✓ **Requirements Management** — Reproducible dependencies
- ✓ **Environment Secrets** — .env for sensitive credentials
- ✓ **Documentation** — Clear, beginner-friendly guides

Future Enhancements

Possible improvements to expand the project:

Incremental Loading — Update tables instead of append-only

Data Validation Framework — Great Expectations for data quality

Automated Testing — CI/CD pipeline (GitHub Actions)

Visualization Dashboard — Plotly/Streamlit interactive dashboard

Cloud Deployment — Airflow/Prefect for scheduling

Advanced Analytics — ML model (e.g., fare prediction)

How to Use This Repo

For Job Applications

1. Review the **project structure** — shows organization & best practices
2. Read **ARCHITECTURE.md** — demonstrates system design thinking
3. Check **src/** code — production-quality, well-commented
4. Run the **notebooks** — executable proof of concept
5. Review **SQL queries** — shows analytical thinking

For Learning

1. Start with **notebooks/00_project_overview.ipynb**
2. Follow the ETL stages: Extract → Transform → Load
3. Explore **sql/analysis/** for real-world query patterns
4. Customize with your own data sources

For Contributing

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/your-feature`
3. Make changes and test: `pytest tests/`
4. Submit a pull request with clear description

License

MIT License — See [LICENSE](#) for details.

Author

Nikitha Kurapati

- Email: nikithachowdarykurapati@gmail.com
- LinkedIn: linkedin.com/in/kurapati-nikitha
- GitHub: github.com/nikitha-k11

Acknowledgments

- **Data Source:** NYC Yellow Taxi Trip Records via Kaggle
 - **Database:** Neon PostgreSQL (free tier)
 - **Inspiration:** Real-world ETL patterns in production systems
-

Notes for Recruiters

This project showcases:

- **Full-cycle data engineering** — Extract, clean, load, analyze
- **Production-ready code** — Modular, logged, tested
- **Real data at scale** — 500K+ rows, actual NYC taxi data
- **SQL expertise** — Complex aggregations, analytics
- **Problem-solving** — Handled storage limits, data quality
- **Documentation** — Clear, professional, job-ready

Perfect for roles in **Data Engineering**, **Data Science**, **Analytics Engineering**, or **Backend Development**.